Number system converstion(bit-binary digit)

Basic python till Bit wise operator_24_&_25-02-2026

In [1]:
```
25
```
Out[1]:
```
25
```

In [2]:
```
bin(25)
```
Out[2]:
```
'0b11001'
```

In [3]:
```
int(0b11001)
```
Out[3]:
```
25
```

In [4]:
```
bin(30)
```
Out[4]:
```
'0b11110'
```

In [5]:
```
int(0b11110)
```
Out[5]:
```
30
```

In [6]:
```
int(0b11001)
```
Out[6]:
```
25
```

In [7]:
```
oct(25)
```
Out[7]:
```
'0o31'
```

In [8]:
```
int(0o31)
```
Out[8]:
```
25
```

In [9]:
```
int(0b11110)
```
Out[9]:
```
30
```

In [10]:
```
0o31
```
Out[10]:
```
25
```

In [11]:
```
0b11001
```
Out[11]:
```
25
```

In [12]:
```
int(0b11001)
```
Out[12]:
```
25
```

In [13]:
```
bin(7)
```
Out[13]:
```
'0b111'
```

In [14]:
```
oct(25)
```
Out[14]:
```
'0o31'
```

In [15]:
```
0o31
```
Out[15]:
```
25
```

In [16]:
```
int(0o31)
```
Out[16]:
```
25
```

In [17]:
```
hex(25)
```
Out[17]:
```
'0x19'
```

In [18]:
```
0x19
```
Out[18]:
```
25
```

In [19]:
```
hex(16)
```
Out[19]:
```
'0x10'
```

In [20]:
```
0xa
```
Out[20]:
```
10
```

In [21]:
```
0xb
```
Out[21]:
```
11
```

In [22]:
```
hex(1)
```
Out[22]:
```
'0x1'
```

In [23]:
```
hex(25)
```
Out[23]:
```
'0x19'
```

In [24]:
```
0x19
```
Out[24]:
```
25
```

In [25]:
```
0x15
```
Out[25]:
```
21
```

swap 2-variable in python

(a,b = 5,6) After swap we should get ==> (a,b = 6,5)

In [26]:
```
a = 5
b = 6
```

In [27]:
```
a = b
b = a
```

In [28]:
```
print(a)
print(b)
```

```
6
6
```

In [29]:
```
# in above scenario we lost the value 5
a1 = 7
b1 = 8
```

In [30]:
```
temp = a1
a1 = b1
b1 = temp
```

In [31]:
```
print(a1)
print(b1)
```

```
8
7
```

- in the above code we are using third variable
- in interview they might ask can we swap better way without using 3rd variable

In [34]:
```
a2 = 2
b2 = 6
```

In [35]:
```
#swap variable formula without using 3rd formula
a2 = a2 + b2 # 5 + 6 = 11
b2 = a2 - b2 # 11 - 6 = 5
a2 = a2 - b2 # 11 - 5 = 6
```

In [36]:
```
print(a2)
print(b2)
```

```
6
2
```

In [37]:
```
0b110
```

Out[37]: 6

In [38]:
```
0b101
```

Out[38]: 5

In [39]:
```
print(0b110)
print(0b101)
```

```
6
5
```

In [40]:
```
print(0b101)
print(0b110)
```

```
5
6
```

In [41]:
```
#but when we use a2 + b2 then we get 11 that means we will get 4 bit which is 1
print(bin(11))
print(0b1011)
```

```
0b1011
11
```

- XOR - there is other way to work using swap variable also which is XOR because it will not waste extra bit

In [42]:
```
print(a2)
print(b2)
```

```
6
2
```

In [43]:
```
#there is other way to work using swap variable also which is XOR because it wil
a2 = a2 ^ b2
b2 = a2 ^ b2
a2 = a2 ^ b2
```

In [44]:
```
print(a2)
print(b2)
```

```
2
6
```

In [45]:
```
a2, b2
```

Out[45]:
```
(2, 6)
```

In [46]:
```
a2, b2 = b2, a2 #how it work is b2 6 a2 is 5 first it goes into stack & then it
```

In [47]:
```
print(a2)
print(b2)
```

```
6
2
```

BITWISE OPERATOR

- We have 6 operators complement (~) || AND (&) || OR (|) || XOR (^) || LEFT SHIFT (<<) || RIGHT SHIFT (>>)
  - Complement (~)

- And (&)
- Or (|)
- XOR (^)
- Left Shift (<<)
- Right Shift (>>)

In [48]:
```
print(bin(12))
print(bin(13))
```

```
0b1100
0b1101
```

In [50]:
```
0b1101
```

Out[50]:
```
13
```

In [51]:
```
0b1100
```

Out[51]:
```
12
```

Complement (~)

In [52]:
```
~12 # we get the original value + one with negative value
```

Out[52]:
```
-13
```

In [53]:
```
~46
```

Out[53]:
```
-47
```

In [54]:
```
~54
```

Out[54]:
```
-55
```

In [55]:
```
~10
```

Out[55]:
```
-11
```

Bitwise Operator

In [56]:
```
12 & 13
```

Out[56]:
```
12
```

In [57]:
```
12 | 13
```

Out[57]:
```
13
```

In [58]:
```
1 & 0
```

Out[58]:
```
0
```

In [59]:
```
1 | 0
```

Out[59]: 1

In [60]:
```
bin(13)
```

Out[60]: '0b1101'

In [61]:
```
print(bin(35))
print(bin(40))
```

```
0b100011
0b101000
```

In [62]:
```
35 & 40
```

Out[62]: 32

In [63]:
```
35 | 40
```

Out[63]: 43

In [64]:
```
# in XOR if the both number are different then we will get 1 or else we will get
12 ^ 13
```

Out[64]: 1

In [65]:
```
print(bin(25))
print(bin(30))
```

```
0b11001
0b11110
```

In [66]:
```
25 ^ 30
```

Out[66]: 7

In [67]:
```
bin(7)
```

Out[67]: '0b111'

In [68]:
```
bin(25)
```

Out[68]: '0b11001'

In [69]:
```
bin(30)
```

Out[69]: '0b11110'

In [71]:
```
0b00111
```

Out[71]: 7

In [72]:
```
bin(10)
```

Out[72]: '0b1010'

In [73]:
```
10<<1
```

Out[73]: 20

In [74]:
```
10<<2
```

Out[74]: 40

In [75]:
```
bin(10)
```

Out[75]: '0b1010'

In [76]:
```
10>>1
```

Out[76]: 5

In [77]:
```
10>>2
```

Out[77]: 2

In [79]:
```
10>>6
```

Out[79]: 0